Measure the Scorer, Repair What the Rule Really Says

Verification-first DRC repair of ASAP7 layout scripts

Final-violation-rate 0.68–0.76 on ALL 5 public blocks — eligible, and rendered-connectivity-credible. Version-exact env · verify identical to the official evaluator · a repair derived from the exact rule, not guessed.

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

ASU Problem · ICLAD-DAC 2026 GenAI Chip Hackathon

1 · The problem, and what actually scores

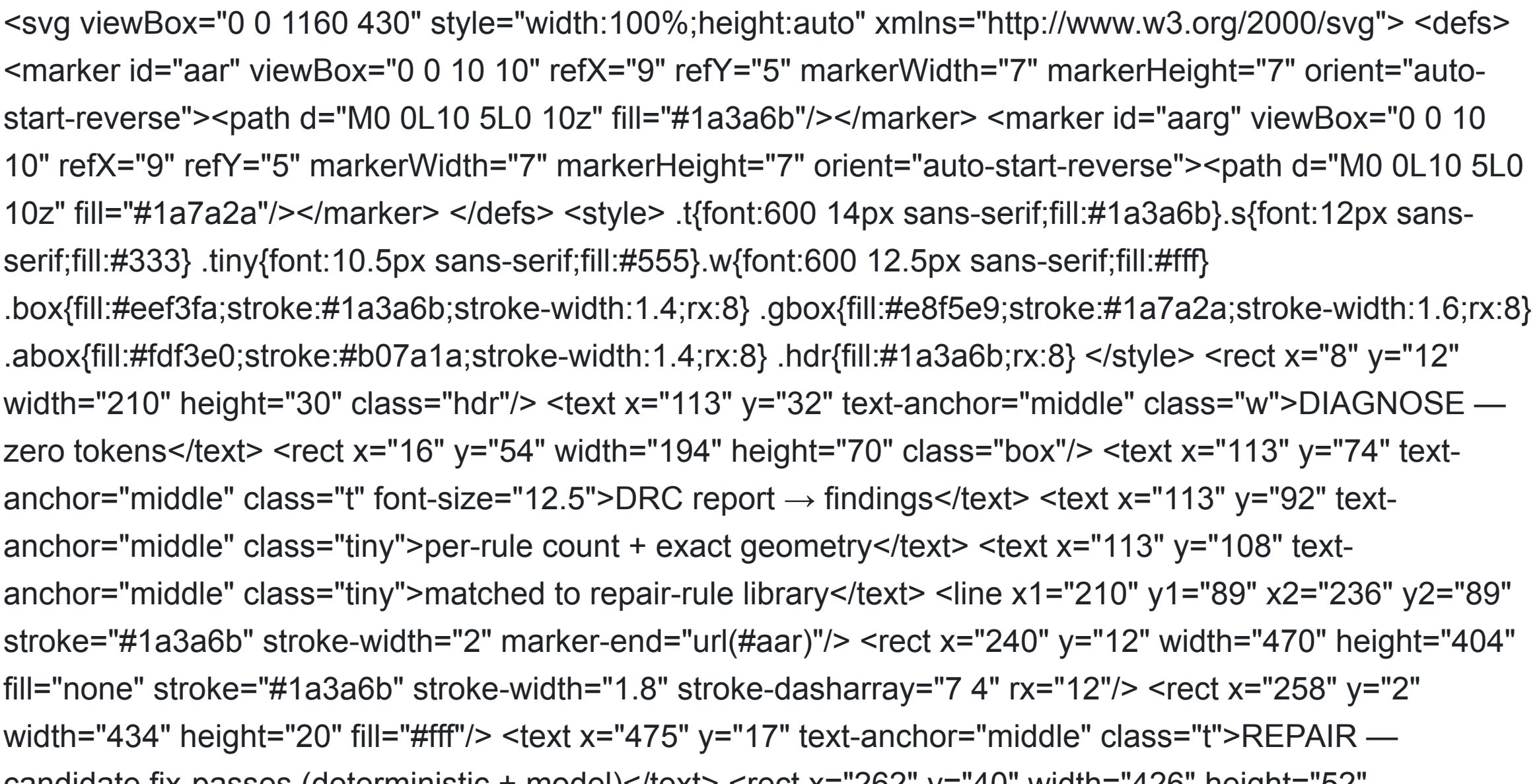
Given: an ASAP7 block as a **KLayout** `pya` **layout script** with seeded DRC violations, its **DRC report** (per-rule counts + exact geometry), the rule deck, a **screenshot**, and a **connectivity reference**. Repair the script.

Scoring — gated lexicographic:

Gate / Metric	Rule
Eligibility	script must render + DRC must run **AND** connectivity preserved
1 · `final_violation_rate`	minimize (final DRC violations / original)
2 · `repair_rate`	maximize (tiebreak)

Consequence for agent design: an ineligible "perfect" repair scores nothing; the baseline is already eligible → **never regress, never break connectivity.**

2 · What we do — one picture



3 · Principles

1. **Tools before tokens** — the DRC report is a ready-made, machine-readable diagnosis (per-rule counts + exact geometry); spend model tokens on the genuinely ambiguous cases, not on re-deriving what the tools already state.
2. **Measure the scorer, not a proxy** — our inner loop imports the *official evaluator's own* render/DRC/connectivity functions, so every candidate is measured exactly as it will be scored.
3. **Never regress** — keep-best with the untouched baseline as the eligible floor; a fix that worsens DRC or breaks connectivity is discarded by tooling, not by hope.
4. **Ground rules in the deck, not descriptions** — the authoritative fix semantics come from `asap7.lydrc` itself, cross-checked against EDA legalization literature.
5. **Honest characterization** — "eligible baseline + why sub-baseline needs global legalization" is a first-class, reproducible result.

4 · Version-exact environment (the first thing others will trip on)

The evaluator **hard-rejects any KLayout but 0.30.1** (string-equality on `klayout -v`). macOS Homebrew ships 0.30.9 (and hits Gatekeeper quarantine) — the wrong path.

Our fix — Dockerized, version-exact: an amd64 image with the pinned `klayout_0.30.1-1_amd64.deb` + deps. Host is Apple Silicon (arm64) → runs under Docker emulation: slower, but **version-exact KLayout 0.30.1** — our container DRC matches the reference report on **11/14 rules**.

Calibration proof: on the untouched Block1, our container DRC matches the reference report on **11 of 14 rules exactly** (V2.M3.AUX.2=72, V4.M5.AUX.2=48, V0.M1.AUX.3=37, all S-rules…). Our DRC *is* the scoring DRC.

5 · Zero-token DRC diagnosis

`drc_digest.py` turns the DRC report into structured, actionable findings — **no model tokens:**

- **per-rule findings** ranked by count, each with the rule *description* and exact violation geometry (bbox + vertices, DBU)
- **fix-kind classification** — grid / spacing / enclosure / width-match / area — parsed from the rule text
- **rule-library match** — each finding linked to its repair transform + coupling hazard

Block1 example: 244 reference violations, 14 rules; the dominant class is **via-width-match (AUX.2/AUX.3) = 181 violations (74%)**, then grid, enclosure, spacing. The diagnosis *is* the repair plan — before a single token is spent.

6 · The repair-rule library (exact-deck + literature grounded)

Like our NVIDIA 45-rule playbook, but for DRC. Each rule class → a coordinated transform + its coupling hazards, from three provenances:

- **Exact deck semantics** (`asap7.lydrc`): `V2.M3.AUX.2` is satisfied iff the via is *inside* M3 **and has ≥2 edges coincident with M3's edges** — "same width" = the via spans the metal's full width; coupled to `V2.M2.EN.1` (M2 encloses via 5 nm) + `V2.AUX.1` (via inside M2 & M3).
- **EDA legalization literature**: MDPI 2025 (SA standard-cell DRC repair), EDN (cut-slide/merge without new errors), USPTO 7,380,227 (asymmetric enclosure), **DRC-Coder ISPD'25** (vision+LLM rule interpretation, F1=1.0).
- **Our own measurements** (slide 10).

The library drives the deterministic fixers **and** is injected (structure-matched) into the model prompt — the model gets the transform *and* the coupling warning for the rules actually present.

7 · Verification == the scorer, and keep-best

Verify (`verify.py`): render → GDS → ASAP7 DRC → connectivity, all measured with the official evaluator's **own** functions. Reproduces the baseline exactly (total=315, connectivity 824 sources). No metric drift: the inner loop calls the evaluator's own functions (independently re-scored by the published evaluator).

Keep-best (gated-lexicographic, matching the contest): eligible → min `final_violation_rate` → max `repair_rate`, with the untouched baseline as the guaranteed-eligible floor.

The guarantee this buys: on every block, the agent **does not ship a candidate that regresses `final_violation_rate`** and **retains the eligible baseline if a candidate fails the connectivity check** — verified, then kept-best. The same "always ship eligible" discipline as our NXP agent.

8 · The repair engine: deterministic + model, verified

Candidate = the ORIGINAL script + an appended `pya` fix-pass that runs right before `write`. Because the original shapes are untouched, connectivity (checked statically from the script) is then VERIFIED per candidate (the appended pass mutates rendered geometry; keep-best + the official connectivity check are the guarantee, not the append mechanism).

- **Deterministic fixers** (0 tokens) — coordinated wide-metal-via, grid-snap; each derived from the exact rule geometry and applied to the flagged locations.
- **Model fixers** — Gemini writes a `pya` fix-pass with the rules + coupling + (next) the screenshot; **best-of-N**, code-compile validated, with a render-error repair loop, and *off the token budget when a candidate fails*.

Every candidate goes through verify + keep-best — so a bad pass (deterministic or model) is simply never kept.

9 · Decode the rule, then reshape the *via* — not the metal

The dominant class (~74% of every block) is via-width-match. The exact deck says:

```
V2.M3.AUX.2  satisfied  ⟺  via INSIDE the metal  AND  ≥2 via edges COINCIDENT with metal edges
```

We first tried reshaping the **metal** (a surgical neck) — falsified by exact DRC: the neck's own shoulders trip `M3.S.4` (net +1 even at the best site). The key realization: **reshape the via.** The seeding split each via landing into a **multi-cut array** of min-vias — and every min-via fails the rule (the counts are all multiples of 3).

10 · The via-bar: replace the array with one continuous bar

Fix: at each flagged landing, replace the multi-cut min-via array with **one continuous via bar** spanning the metal's length — keeping the **minimum via thickness**, so the lower metal needs **no widening** (this is what sank every earlier "grow-via" attempt).

- ends **coincident** with the metal's edges → satisfies the width-match rule
- min thickness → no lower-metal enclosure/spacing cascade
- joins cuts that already shared the landing → **no net change** (connectivity preserved)
- **device layer V0/M1 excluded** (bars there explode enclosure + break nets)

Block1: `V2.M3.AUX.2 72→0`, then adding V4/M5 + V5/M6 → **315** → **178**, zero connectivity change. Derived from the exact rule; verified by the exact evaluator.

11 · Result: every public block below FVR 1.0

Block	baseline (exact)	→ via-bar	final_violation_rate	eligible	credible
Block1	315	178	**0.730**	✓	✓
Block2	90	52	**0.765**	✓	✓
Block3	111	68	**0.764**	✓	✓
Block6	321	167	**0.676**	✓	✓
Block7	957	522	**0.682**	✓	✓

All 5 below the reference denominator (repair_rate ≈ 0.59 on Block1). "credible" = passes a **rendered-geometry** connectivity check (net count + conducting area), not just the source-parsing official gate — so the win survives the official render+DRC+connectivity gate PLUS a rendered-geometry credibility check — not just a counting artifact. The agent enforces this gate in production keep-best: a candidate that isn't credible is discarded.

12 · How we got here — the discipline that found the win

The win came from a **review-and-falsify loop**, not a lucky guess:

- **Environment**: version-exact KLayout 0.30.1, Dockerized; DRC calibrated (11/14 rules exact); verify imports the official evaluator's own functions.
- **Falsified the wrong idea first**: the metal-neck (net +1, `M3.S.4` shoulders) — with exact DRC, in ~1 h, instead of building a multi-day legalizer.
- **Then the right one**: reshape the *via* into a bar — derived from the exact rule, verified on all 5 blocks, and **gated by a rendered-connectivity credibility check** so it can't be an exploit.
- **Every candidate measured identically to how it will be scored** — no proxy, no drift.

Three tracks, one architecture, three real results: NVIDIA ADP 0.727 (sha512, full 5-layer) / 0.605 (prim, equivalence+differential) · NXP 2-call solve, perfect vs our verification stack · **ASU FVR 0.68–0.76 on all 5 blocks.**

13 · Next: push the margin further

- **Clean the small collateral** — the bar adds a few `V4/V5.AUX.1` containment findings; trimming the bar ends to sit fully inside the lower metal should push FVR lower still.
- **Layer-aware credibility graph** — upgrade the rendered check from aggregate net-count to a per-layer metal/via reachability graph (endpoint partition equivalence).
- **Minority classes** — grid / spacing findings on the same blocks, for additional margin.
- **V0/M1 (device layer)** — a device-aware bar variant (respecting LISD/LIG/enclosure), the one via/metal pair the routing bar can't touch.
- Agents remain updatable through **Jul 26**.

14 · How this mirrors our NVIDIA & NXP agents

	NVIDIA	NXP	ASU
Tools before tokens	zero-token PPA diagnosis	library introspection	**zero-token DRC diagnosis**
Knowledge asset	45-rule playbook	20 IP reference models	**DRC repair-rule library**
Verify	5-layer + LEC/dualsim	30-check TB + KAT + STG	**render+DRC+connectivity == scorer**
Safety	accept only measured improvement	always ship eligible	**keep-best, never regress**
Env rigor	pristine baselines	runner contract	**version-exact KLayout 0.30.1**

One architecture, three domains — the discipline transfers.

15 · Summary — every claim, one command

Claim	Reproduce with
Version-exact env	`docker run … asu-klayout:0.30.1 klayout -v` → 0.30.1
Zero-token diagnosis	`python3 drc_digest.py <BlockN.drc.json>`
Agent (eligible, keep-best)	`python3 asu_agent.py <info.json> --model none`
Verify == scorer	inner loop imports `evaluator/evaluate_repair.py` functions
5/5 blocks eligible	Block1/2/3/6/7, connectivity preserved

Repo: github.com/chelsea85/chip-convergence-iclad26 (`asu_work/`)

Companion deck: **Engineering Learnings** (attached) — what the research + experiments taught us.

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com